

E-Ring Voting — Explain Like I’m Five

A walk-through with no math, for a curious high-schooler

Daniele Lin

Niccolò Pagano

May 2026

This document explains a small computer-science research project we built. It is written for a smart high-school student who has done a bit of programming and is curious about how online voting could actually work. There are no equations in this document — just analogies, a few diagrams, and a careful build-up of the ideas. If you want the mathematical version, the file `notes.tex` in the same folder covers the same material as a university-level walk-through.

By the end of this you should be able to explain to a friend what the project does, why it is harder than it sounds, and what we think we got right and wrong.

Contents

1	What’s the point?	2
2	The contradiction, in concrete terms	2
3	The magic envelope: ring signatures	3
4	The double-vote problem	3
5	Where do you write everything down?	4
6	Who is allowed to write on the board?	5
6.1	Option A: a small named team (the “federated” answer)	5
6.2	Option B: anybody can write (the “blockchain” answer)	5
6.3	The honest comparison	6
7	What this project actually built	6
8	What this project did <i>not</i> fix	7
9	How this connects to things you may have heard of	7
10	Glossary	8

1 What's the point?

We built a small system for running **anonymous online votes** that also lets anyone in the world **check the result**. Think of a class president election, a board vote in a company, or a small referendum — not a national election.

Two properties have to be true at the same time:

- **Nobody can tell how you voted.** Not the people running the election, not the developers, not anyone who reads the public record of the vote.
- **Anyone can recompute the result.** Given the public record of the election, you can do the count yourself, with your own laptop, and refuse to accept any other claimed result.

If you stop and think about it for a minute, those two properties look like they contradict each other. “The votes are secret” usually means “trust us, you can’t see them.” “Anyone can recompute” usually means “the votes are public.” How can both be true?

The whole project is built around one mathematical trick that breaks the contradiction. The rest of these notes explain the trick (in words), what we built around it, and where the limits are.

2 The contradiction, in concrete terms

Imagine your class is voting on lunch. The options are pasta, pizza, *focaccia*. You write your choice on a slip of paper and drop it in a sealed box. At the end of the period, the teacher opens the box and counts.

What can go wrong?

- **The teacher cheats on the count.** You can’t tell, because you’re not the one counting. Maybe the teacher swaps two ballots, or pretends one didn’t exist.
- **The teacher peeks at who voted for what.** You can’t tell, because they own the box.
- **Someone votes twice.** The teacher might or might not notice; you certainly can’t.

The naive online version is worse. The website stores your vote in a database. The database administrator can read it. The server can lie about the count. You have to trust a stranger to be honest, and even if they are honest, you have to trust they wrote the code right.

What we want is a system where:

1. Your ballot, the very moment you cast it, is mixed in with everyone else’s so well that even the people running the election can’t tell which one is yours.
2. Every ballot is written down in a public place that anyone can read.
3. The rules of the system make it *impossible* for a cheater to publish a fake count without it being publicly obvious.

Property 1 sounds the most magical. Property 2 sounds easy. Property 3 sounds nice but vague. The next sections build up each one.

3 The magic envelope: ring signatures

Here is the key idea, in an analogy.

The crowd analogy.

Suppose 50 people are in a room. One of them stands up and shouts “*I am one of the 50 people in this room.*” You believe them — they can prove it by, say, holding up a list of all 50 names including their own. But there is no way to tell *which* of the 50 it was. They could be any of them.

A **ring signature** is the digital version of that. It is a way to sign a message such that the signature proves “*one of these 50 people signed this,*” without revealing which one.

A real-world example you have probably encountered without realising: when a whistleblower leaks something to a journalist and the journalist says “a source inside the company,” the source is anonymous within the crowd “employees of the company.” If the company has 5000 employees, the crowd is big enough to hide in. If the company has 3 employees, it isn’t.

So if every voter in our election has a digital “signature key” (a secret number only they know), and every voter publishes their “signature lock” (a public number that proves who they are), then we can build the ballot like this:

- Alice wants to vote for pasta.
- She uses her secret key to produce a *ring signature* on the word “pasta.” The ring is *everyone’s public locks*, including hers, mixed together.
- Anyone in the world can check the signature: it is valid, so *one of the 50 registered voters* voted for pasta.
- Nobody can tell it was Alice. Not even the teacher.

This is property 1 from before. The ballot is publicly visible, but the *voter* is hidden inside the crowd of registered voters.

Why this is mathematically possible.

The math behind ring signatures uses something called *public-key cryptography* — the same family of math that protects your bank website (the little padlock in the browser). The trick is that with the right kind of math, you can build a signature that depends on *at least one* person’s secret key, but doesn’t reveal *which* one. From the outside, all 50 possible signers look equally likely to have produced it.

The exact construction we use was proposed in 2021 by two researchers, Alessandra Scafuro and Bihan Zhang. The repository implements their recipe step by step.

4 The double-vote problem

If the ballot hides the voter, what stops Alice from voting twice?

Plain ring signatures don’t stop her. She could sign “pasta” once, then sign “pizza” again, and there would be no way to tell that both signatures came from the same person.

The fix is a feature called **one-time traceability**, which is what the “T” in our project name (e-Ring-voting + one-time **T**raceable) refers to. Here is the analogy:

The wax-seal analogy.

Imagine that when you sign a ballot, the signature automatically imprints a unique seal — like a wax stamp — that depends only on *you* and *this election*. The seal is the same every time *you* sign in *this* election, no matter what the ballot says. But the seal is also *cryptographically scrambled* so that nobody can read it as “this is Alice’s seal” — it just looks like random noise.

Now consider what happens if Alice signs two different ballots (“pasta” and “pizza”):

- The two signatures have the same scrambled seal — because both came from Alice in this election.
- Anyone comparing the two ballots can see “hey, these have a matching seal — the same person produced both.”
- And, by a bit of further math, the matching seal actually reveals *which* voter (which public lock) it belongs to. So Alice is publicly named as the cheater.

But if Alice signs only *once*, her seal is on exactly one ballot. No other ballot has it. So the seal stays scrambled, unreadable, and Alice remains anonymous inside the 50-voter ring.

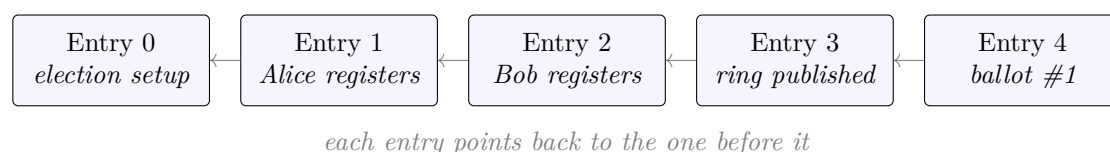
This is the cleverest part of the whole system. The same signature both *protects* you when you behave honestly and *exposes you* the moment you try to cheat.

Watch out.

This trick works *only within one election*. A different election has a different seal-recipe, so Alice’s seal looks completely different from one election to the next. You cannot use it to track a voter across elections.

5 Where do you write everything down?

Anonymity (the ring signature) and one-vote-per-person (the traceable seal) are the cryptographic core. But for an auditor to actually *check the count*, the ballots have to live somewhere public. We call this public place the **bulletin board**. Think of it as a notice board in the middle of a town square. Anyone can read it. Nobody can erase entries. Every new entry shows the time it was posted. And every entry has a tiny pointer to the previous entry, so you can scroll back through the entire history.



The key property: **if anyone tries to silently edit an old entry, the pointer chain breaks.** The whole point of the chain is to make tampering loud — a single changed byte invalidates every entry that came after.

The state of the election goes through phases on the board:

1. **Setup.** The election parameters are written: options (pasta / pizza / focaccia), the schedule (when voting opens and closes), and who is allowed to run the election.
2. **Registration.** Each voter’s public lock is added. After registration closes, the list of voters is frozen.
3. **Ring.** The frozen list is published as “the ring” — the official anonymity crowd. From now on, ring signatures from anyone in this crowd are valid ballots.
4. **Ballots.** Voters cast their ring-signed ballots. Each one gets a new line on the bulletin board.
5. **Closed.** When the schedule says voting is over, the bulletin board is sealed.
6. **Tally.** The organisers publish their count — but crucially, anyone can recompute it from the public ballots and contradict them if they cheat.

6 Who is allowed to write on the board?

This is where the project gets interesting, because there are two very different sensible answers, and we implemented *both* so we could compare them.

6.1 Option A: a small named team (the “federated” answer)

A small group of named people — say, 3 trusted scrutineers, publicly identified by name — jointly run the bulletin board. To write a new entry, at least 2 of the 3 have to digitally co-sign it.

- **Good news:** fast. No expensive computation. Everyone knows who they are.
- **Good news:** a corrupt minority can’t cheat. If scrutineer 1 alone tries to add a fake ballot, scrutineers 2 and 3 refuse to sign and the entry is invalid.
- **What if two of them go bad together?** We add another layer: a separate group of *witnesses* (say, three more named observers) who occasionally take a snapshot of the board and sign it. If the two corrupt scrutineers tried to show “ledger version A” to one witness and “ledger version B” to another, the two witnesses would later disagree on what they saw, and the disagreement is mathematical proof of cheating.

This is the same family of system as the one that secures HTTPS certificates on the web (“Certificate Transparency”), and the same shape as Sigsum (used for signed software releases). It is a well-known pattern for keeping known parties honest.

6.2 Option B: anybody can write (the “blockchain” answer)

The other answer is the Bitcoin-style answer: don’t pick a small named team at all. Instead, *anyone* on the internet can compete to add the next entry. The competition is solving a small puzzle (the “proof of work”): basically, your computer guesses random numbers until one of them, when hashed, has enough leading zeros. The first person to find a good number gets to add the next block of ballots, and gets paid a small reward.

Why the puzzle?.

The puzzle exists to make it expensive to take over the system. If adding entries were free, anyone could pretend to be a thousand different identities at once and stuff the chain with fake entries. The puzzle forces each “identity” to do real computational work, which costs electricity, which costs money. So a faker has to pay for every block they add.

- **Good news:** no named team. Anyone with a computer can participate. No genesis-pinned trust.
- **Bad news:** the puzzle is slow (blocks take minutes, not seconds). The system runs an internal mini-currency (“eVotes”) to pay the puzzle-solvers.
- **Bigger bad news:** for an election that matters politically, an attacker can *rent* a huge amount of computer power online (services like NiceHash sell hashrate by the hour). If their incentive — e.g., swinging the outcome of a corporate-board election worth millions of dollars — is bigger than what it costs to rent the computing power for the voting window, they win. This is the argument that Park and Specter (MIT, 2021) made in a famous paper called *Going from bad to worse: from internet voting to blockchain voting*.

6.3 The honest comparison

	Option A (named team)	Option B (open mining)
Who can write?	A known team (e.g., 2-of-3 scrutineers)	Anyone with a computer
How fast?	Seconds	Minutes (slow on purpose)
Operating cost?	Just the team’s time	Electricity for the puzzle
Built-in currency?	No	Yes (otherwise nobody mines)
Best protection against?	A corrupt minority of the team	A corrupt minority of miners
Weakest if?	The team unanimously corrupts	Someone rents enormous hashpower

Neither is universally better. The named-team option works when the election has named accountable scrutineers (a school senate, a co-op board, an inter-institution committee). The blockchain option works when there is genuinely *no* group anyone trusts to be the team — even a small one. The cost of that second case is that you have to defend against rented-hashpower attacks, which gets *harder* the higher the political stakes.

7 What this project actually built

We built both options. The repository is split into three folders:

- `otrs/` — the cryptography (the magic envelope + the seal trick). About 1,100 lines of Python. Used by both options below.
- `voting/` — Option A, the named-team bulletin board. About 1,800 lines.
- `chain/` — Option B, the proof-of-work chain. About 1,400 lines.

Plus 127 automated tests that check the whole thing actually works the way we say it does.

We also ran benchmarks to compare. At 50 voters, the named-team option finishes a complete election in about 1.1 seconds and produces a 324 KB public record. The blockchain option finishes the same election in about 3.0 seconds and produces a 344 KB record — so it is roughly 1.6× slower and uses about 6% more storage, mostly because the proof-of-work puzzle has to be solved for every block.

Both options produce the same answer to the question “who won.” The cryptography that makes the votes anonymous and catchable-if-double-counted is exactly the same; what differs is who gets to write on the public record.

8 What this project did *not* fix

It’s important to be honest about what we did not solve. These are real, hard, open problems — not ones a small project can knock out.

- **Coercion.** If someone threatens you and says “vote for pasta or I’ll fire you,” you might want to prove to them that you obeyed. With our system, you actually *can* prove it (by revealing your seal-recipe), so a coercer can force you to comply. Fixing this is called *coercion resistance* and requires significantly more machinery (a 2005 paper called JCJ is the canonical starting point). It is one of the most active open problems in voting research.
- **Compromised computers.** If a virus on your laptop secretly signs whatever it wants, the cryptography can’t help you — the system can’t tell the difference between “Alice clicked pasta” and “Alice’s malware clicked pasta.” Real deployments need hardware (smart cards, HSMs) and that is a different engineering project.
- **Quantum computers.** The math we rely on can be broken by a sufficiently large quantum computer (the algorithm is called Shor’s). Those computers don’t exist yet at scale, but if they do, our system will need to be re-built on top of different mathematics (“post-quantum cryptography”).
- **Ring size.** Right now, an election with 1000 voters has 1000-person rings, and each ballot’s signature is about 64 kilobytes. That’s fine for small elections; it’s not fine for nation-scale ones. The fix is a class of *logarithmic-size* ring signatures (Triptych, Groth–Kohlweiss-style), which we did not implement.
- **Truly proving the cryptography.** The math is correct in the paper that proposed it, but our specific Python implementation has not been independently audited or machine-checked using formal-verification tools like EasyCrypt. A machine-checked proof would be a research contribution in its own right.

9 How this connects to things you may have heard of

Bitcoin. Bitcoin invented the proof-of-work chain we used for Option B. The difference is what is stored on the chain — Bitcoin stores money transfers, we store election records — and what we use it for. Most relevant to read first: Wikipedia on “proof of work,” then Nakamoto’s 2008 Bitcoin whitepaper if you want the original.

Certificate Transparency. When your browser visits a website, the website shows a certificate proving it owns the domain. Certificate Transparency is a system of public bulletin boards that record every certificate ever issued, so if a bad certificate authority secretly issues a fake

certificate for, say, `gmail.com`, it gets logged publicly and the attack is visible. Option A in our project is the same idea, transplanted to voting.

Voting research. Two systems you might run into: Helios (2008) is one of the earliest practical end-to-end-verifiable voting systems. ElectionGuard (Microsoft, 2021) is more recent. Our project sits in the same family but uses a different building block (ring signatures) and is much smaller in scope.

10 Glossary

Anonymity set

The crowd you are hiding inside. In our case, the set of registered voters in this election.

Ballot

A single voter’s signed choice.

Bulletin board

The append-only public record where every election event is written. “Append-only” means new entries can be added but old entries cannot be edited or removed.

Election ID

A unique label for one specific election. The seal mentioned in the wax-seal analogy depends on it, which is why a voter’s seal looks different in different elections.

Hash

A short fingerprint of a longer piece of data. “SHA-256” is the most common one. Changing even one bit of the input changes the fingerprint completely.

Mining

In Option B, the activity of guessing nonces until the puzzle is solved. The reward for finding a good nonce is paid in eVotes.

Public-key cryptography

Math where you have a public “lock” anyone can use to send you encrypted messages or verify your signatures, and a private “key” only you know that you use to read messages or produce signatures. The web’s HTTPS padlock relies on this.

Ring signature

A signature that proves *one of these n people* signed it, without revealing which one.

Trace tag (“seal” in this document)

The mathematical object that’s the same every time a given voter signs in a given election. Collisions reveal a double-voter.

What to read next

If you want the technical version of the same material, with the actual mathematics, read `notes.tex` in this same folder (25 pages). It walks through the same ideas with formulas, code snippets, and a chapter on why the chain version is harder than it looks.

If you want to skip the implementation details and read the research-style write-up, read `paper/artifact.tex` (16 pages). It is structured like a small academic paper.

If you want to read the code, the easiest starting point is `otrs/otrs.py` — the central file — followed by `tests/test_otrs.py` which checks it works correctly.